Experiential Learning
Project Assessment-I
on
"Random Password Generator in Python"

under the course

Python Programming Language (N-PCCCM304P)

Presented By :
Ayush Rahangdale
[CM25009]

Under the Guidance
of
Mrs. Suchita Meshkar

Session:2026-27 (ODD)

Department of CSE(AI&ML)

S. B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND RESEARCH,NAGPUR
An Autonomous Institute, Affiliated to RTMNU, Nagpur

Problem statement

People often create simple passwords like 123456 or their own names.

Predictable passwords make user accounts easy to hack.

It is hard for users to manually make strong, complex passwords with numbers and symbols.

We need a Python program to generate strong, random, and unguessable passwords automatically.

Objectives

To develop a simple tool that generates secure, random passwords.

To let the user choose the password length.

To allow users to select uppercase, lowercase, numbers, and special symbols.

To ensure at least one character from each selected category is included.

To check and display the password strength rating (Weak, Moderate, Strong).

Algorithm

Start the program.

Ask the user for the password length.

Ask the user which character types to include (upper, lower, digits, symbols).

Combine all selected character sets into one pool.

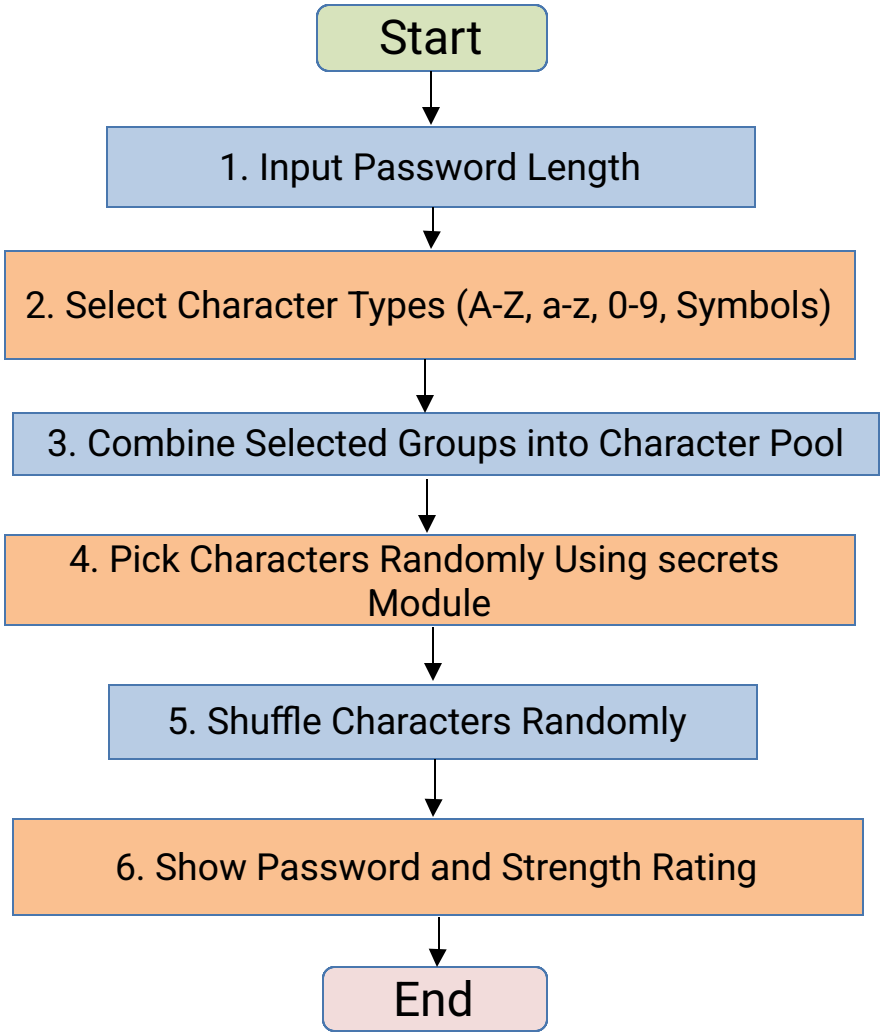
Pick random characters from the pool until the desired length is reached.

Shuffle all characters randomly so the pattern cannot be guessed.

Display the generated password and its strength rating.

Stop the program.

Flowchart/pseudocode



Understanding of Python logic

```python
import secrets
import string

# 1. Build character pool
pool = string.ascii_lowercase
if use_upper:   pool += string.ascii_uppercase
if use_digits:  pool += string.digits
if use_special: pool += "!@#$%^&*()"

# 2. Pick characters securely and join
password = "".join(secrets.choice(pool) for _ in range(length))
```

How it works

Takes Your Choice: You type how long you want the password to be and choose what to add (capital letters, numbers, or symbols).

Makes a Letter Box: The program puts all your chosen letters, numbers, and symbols into one single list.

Picks Randomly: It picks one random character from the list one by one until your password reaches the full length.

Shows the Password: It joins all the picked characters together and shows your new password on the screen.

Benefits

Very Safe: Passwords are fully random, so hackers cannot guess them.

Easy to Change: You can easily make short, long, simple, or complex passwords.

Saves Time: You get a strong password in just one second without thinking hard.

No Bad Habits: Stops people from using easy passwords like their name or 123456.

Fast and Simple: Works instantly without slowing down your computer.

GitHub Details & My Contribution

Repository Name : random-password-generator

My Contribution:

Understood the security requirements and designed the algorithm.
Wrote the clean Python program using secrets and string modules.
Added validation and password strength evaluation.
Created the GitHub repo and prepared presentation slides.

Result / Expected Output

```
============================================
Generated Password(s) | Strength: Strong (Very Secure)
============================================
Password 1: k9#TvB7!mQ2x
Password 2: R4@pL9$wZ1eM
============================================
```

Phase 2

Improvements

Added **uppercase, lowercase, numbers and symbols**

Added **minimum password length checking**

Made sure selected character types are included

Used **secure random generation**

Added **password shuffling**

Added **password strength checking**

Added **Y/N input validation**

Added option to **generate another password**

Added error handling for invalid input

Main Features

1. **Custom Length**

User can choose the password length.

2. Character Selection

User can select uppercase, lowercase, numbers and symbols.

3. Secure Generation

The program uses Python's secrets module.

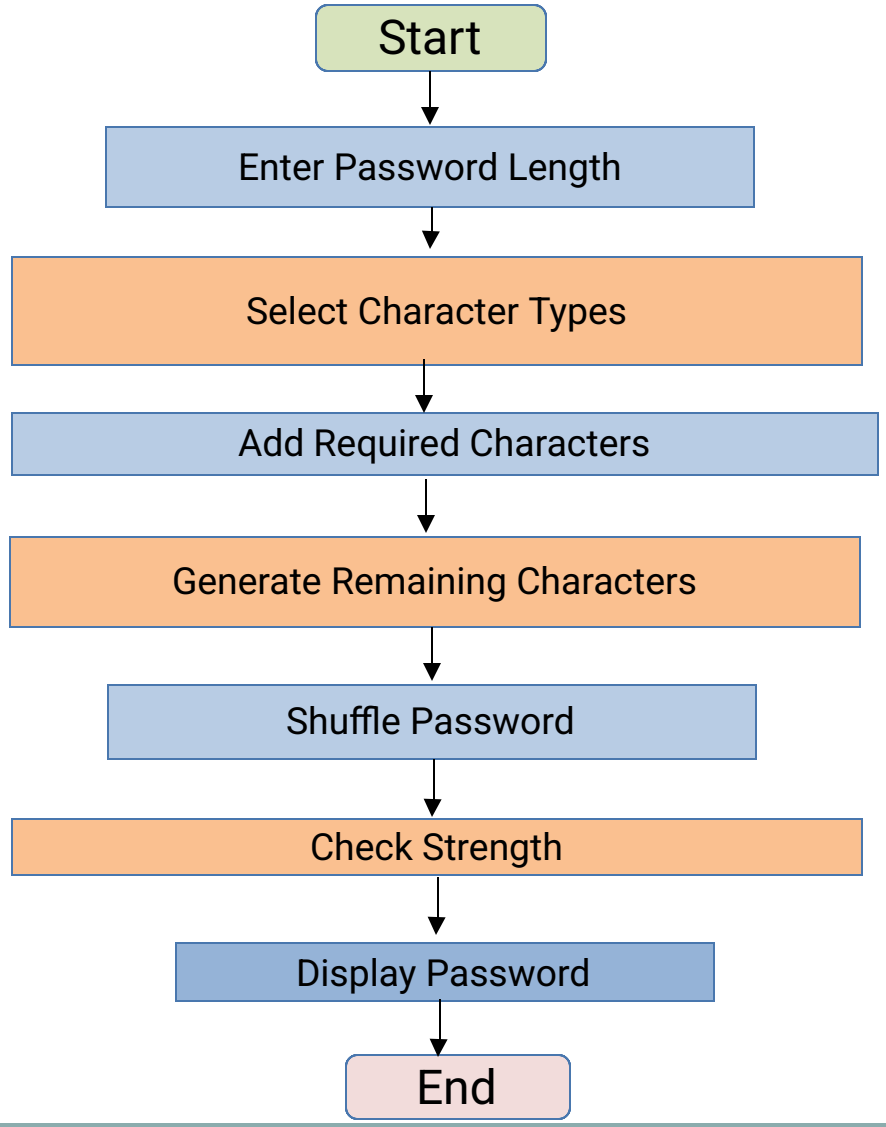
4. Strength Checking

Password is shown as **Weak, Medium or Strong** .

5. Generate Again

User can create another password without restarting the program.

Algorithm


Start
Enter Password Length
Select Character Types
Add Required Characters
Generate Remaining Characters
Shuffle Password
Check Strength
Display Password
End

Code

```python
import secrets
import string


class PasswordGenerator:

    def __init__(self):
        self.uppercase = string.ascii_uppercase
        self.lowercase = string.ascii_lowercase
        self.numbers = string.digits
        self.symbols = "!@#$%^&*()-_=+[]{}<>?"

    def get_character_set(self, upper, lower, numbers, symbols):
        characters = ""

        if upper:
            characters += self.uppercase

        if lower:
            characters += self.lowercase

        if numbers:
            characters += self.numbers

        if symbols:
            characters += self.symbols

        return characters

    def generate_password(self, length, upper, lower, numbers, symbols):

        if length < 4:
            return None, "Password length must be at least 4."

        selected = []
```

```
        if upper:
            selected.append(self.uppercase)

        if lower:
            selected.append(self.lowercase)

        if numbers:
            selected.append(self.numbers)

        if symbols:
            selected.append(self.symbols)

        if not selected:
            return None, "Select at least one character type."

        if length < len(selected):
            return None, "Length is too small for selected character types."

        # One character from every selected category
        password = [
            secrets.choice(character_set)
            for character_set in selected
        ]

        # Remaining characters
        all_characters = self.get_character_set(
            upper, lower, numbers, symbols
        )

        for _ in range(length - len(password)):
            password.append(secrets.choice(all_characters))

        # Secure shuffle
        for i in range(len(password) - 1, 0, -1):
```

```
        for i in range(len(password) - 1, 0, -1):
            j = secrets.randbelow(i + 1)
            password[i], password[j] = password[j], password[i]

        return ''.join(password), "Success"

    def check_strength(self, password):

        score = 0

        if len(password) >= 8:
            score += 1

        if len(password) >= 12:
            score += 1

        if any(c.isupper() for c in password):
            score += 1

        if any(c.islower() for c in password):
            score += 1

        if any(c.isdigit() for c in password):
            score += 1

        if any(c in self.symbols for c in password):
            score += 1

        if score <= 2:
            return "Weak"

        elif score <= 4:
            return "Medium"

```

```
104         else:
105             return "Strong"
106 
107 
108 def get_yes_no(message):
109 
110     while True:
111         answer = input(message).lower()
112 
113         if answer in ['y', 'n']:
114             return answer == 'y'
115 
116         print("Please enter only Y or N.")
117 
118 
119 def main():
120 
121     generator = PasswordGenerator()
122 
123     print("=" * 50)
124     print("       RANDOM PASSWORD GENERATOR")
125     print("=" * 50)
126 
127     while True:
128 
129         try:
130             length = int(input("\nEnter password length: "))
131 
132             upper = get_yes_no("Include uppercase letters? (Y/N): ")
133             lower = get_yes_no("Include lowercase letters? (Y/N): ")
134             numbers = get_yes_no("Include numbers? (Y/N): ")
135             symbols = get_yes_no("Include special symbols? (Y/N): ")
136 
```

```
            password, message = generator.generate_password(
                length,
                upper,
                lower,
                numbers,
                symbols
            )

            if password is None:
                print("\nError:", message)
                continue

            print("\n" + "-" * 50)
            print("Generated Password :", password)
            print("Password Length    :", len(password))
            print("Password Strength  :", generator.check_strength(password))
            print("-" * 50)

            again = get_yes_no("\nGenerate another password? (Y/N): ")

            if not again:
                print("\nThank you for using Password Generator!")
                break

        except ValueError:
            print("\nError: Please enter a valid numeric length.")

        except Exception as e:
            print("\nUnexpected error:", e)


if __name__ == "__main__":
    main()
```

Output

```
==================================================
        RANDOM PASSWORD GENERATOR
==================================================

Enter password length: 12
Include uppercase letters? (Y/N): Y
Include lowercase letters? (Y/N):
Please enter only Y or N.
Include lowercase letters? (Y/N): N
Include numbers? (Y/N): Y
Include special symbols? (Y/N): N

--------------------------------------------------
Generated Password : T59JUK2BA56G
Password Length    : 12
Password Strength  : Medium
--------------------------------------------------

Generate another password? (Y/N):
```

Phase 3

Advanced Password Security Tool | GUI, Passphrase & Entropy

Improvements

Upgraded to a **Tkinter GUI** desktop application

Added **Custom Password** and **Memorable Passphrase** modes

Added **entropy-based strength rating** (Weak to Very Strong)

Added option to **exclude confusing characters** (O, 0, I, l, 1)

Added **Show / Hide password** toggle

Added **Copy to clipboard** and **Clear results** buttons

Added option to generate **1 to 5 passwords** at once

Added **input validation** with error pop-up messages

Used a **class-based (OOP) design** with secure shuffling

Main Features

1. Two Generation Modes
Custom password or memorable passphrase (words + number).

2. Flexible Settings
Length 8-64, 1-5 passwords, four types, exclude O, 0, I, l, 1.

3. Secure Generation
Uses secrets module, one character per type, then shuffle.

4. Strength and Entropy
Weak to Very Strong rating with entropy shown in bits.

5. Easy to Use
Show / Hide passwords, copy to clipboard and clear results.

Libraries & Concepts Used

tkinter / ttk
Builds the GUI: frames, labels, entries, buttons, radio buttons, checkbuttons and listbox.

messagebox
Shows pop-up dialogs for input errors and for the copy confirmation.

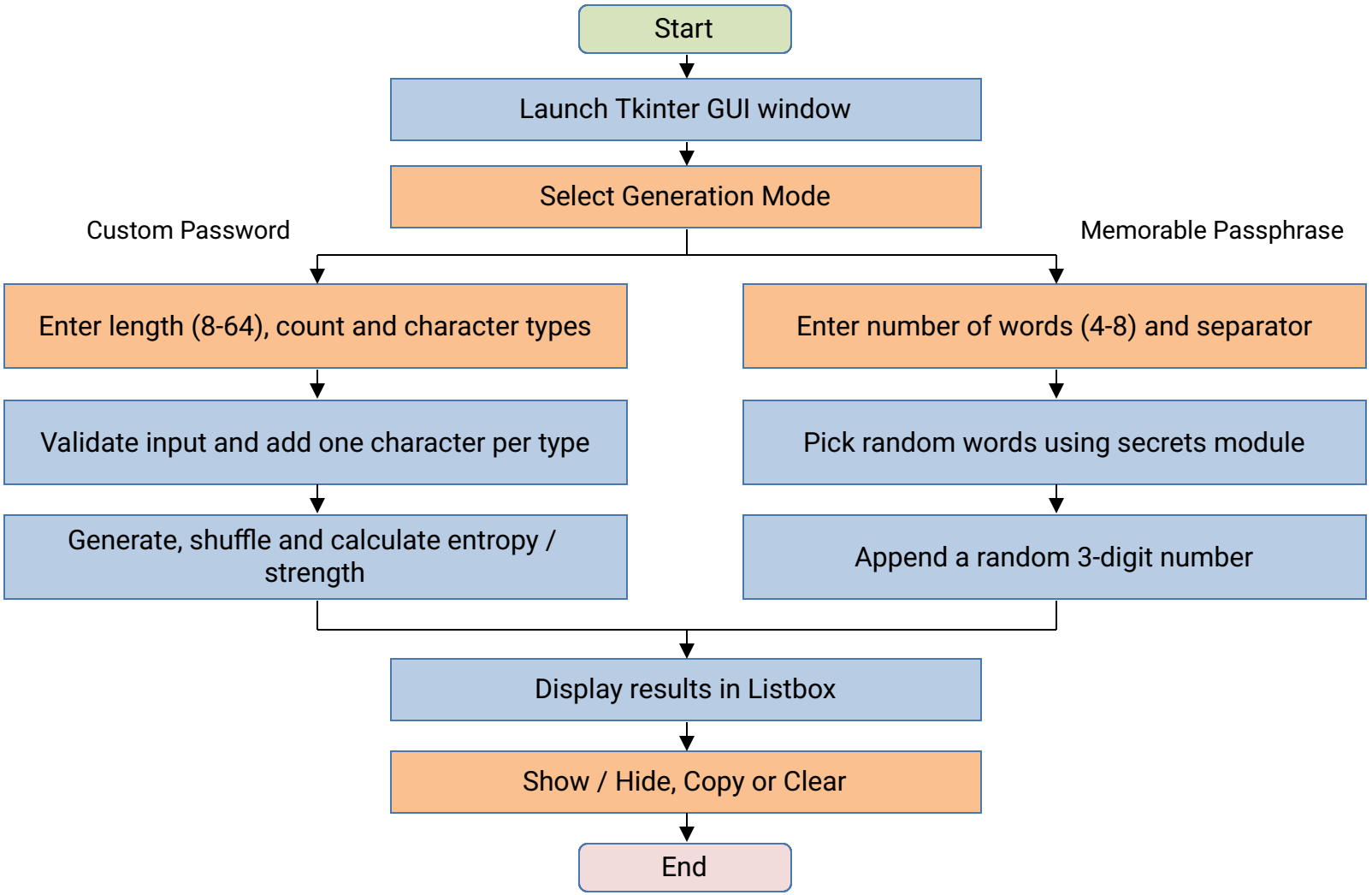
secrets
Cryptographically secure choice, SystemRandom shuffle and randbelow.

string and math
Ready-made character sets for the pool and log2 for entropy.

Classes and OOP
PasswordGeneratorApp keeps state in tk variables and event methods.

Exception Handling
try / except ValueError with range checks for length, count and words.

Algorithm


Start
Launch Tkinter GUI window
Select Generation Mode
Custom Password
Memorable Passphrase
Enter length (8-64), count and character types
Enter number of words (4-8) and separator
Validate input and add one character per type
Pick random words using secrets module
Generate, shuffle and calculate entropy / strength
Append a random 3-digit number
Display results in Listbox
Show / Hide, Copy or Clear
End

Strength & Entropy Logic

$$\text{Entropy (bits)} = \text{Length} \times \log_2(\text{Pool Size})$$

Rating	Rule used in the program
Weak	Length below 8 or entropy below 40 bits
Moderate	Length below 12 or entropy below 60 bits
Strong	Length 12 or more and entropy 60 bits or more
Very Strong	Length 16 or more and entropy 90 bits or more

Example

16 characters, pool of 86 symbols:
$16 \times \log_2(86) \approx 102.8$ bits = Very Strong

Passphrase Mode

Shown as "Not formally assessed" in this phase; only its length is displayed.

Code (Part 1 of 6)

```python
import tkinter as tk
from tkinter import ttk, messagebox
import secrets
import string
import math

SYMBOLS = "!@#$%^&*()-_=+[]{};:,.?/"
AMBIGUOUS = set("O0Il1")

WORDS = [
    "amber", "apple", "arrow", "autumn", "bamboo",
    "beacon", "berry", "bird", "breeze", "cactus",
    "candle", "canvas", "cedar", "cherry", "cloud",
    "comet", "coral", "crystal", "daisy", "delta",
    "desert", "diamond", "dolphin", "dream", "eagle",
    "earth", "echo", "ember", "falcon", "feather",
    "forest", "garden", "glacier", "gold", "harbor",
    "hazel", "honey", "island", "ivory", "jasmine",
    "jungle", "lagoon", "lantern", "lemon", "lotus",
    "maple", "marble", "meadow", "meteor", "mint",
    "moon", "mountain", "nectar", "nebula", "ocean",
    "olive", "orbit", "orchid", "otter", "panda",
    "pebble", "pepper", "phoenix", "planet", "pocket",
    "quartz", "rabbit", "rainbow", "raven", "river",
    "rocket", "shadow", "silver", "sparrow", "star",
    "stone", "summer", "sunrise", "tiger", "timber",
    "topaz", "tulip", "valley", "velvet", "violet",
    "willow", "winter", "wizard", "zephyr"
]


def generate_password(length, categories):
    if not categories:
        raise ValueError("Select at least one character category.")

    if length < len(categories):
        raise ValueError("Password length is too short.")

    password = [
        secrets.choice(category)
        for category in categories
    ]

    pool = "".join(categories)

    while len(password) < length:
        password.append(secrets.choice(pool))

    secrets.SystemRandom().shuffle(password)

    return "".join(password), len(set(pool))


def check_strength(password, pool_size):
    entropy = len(password) * math.log2(pool_size)

    if len(password) < 8 or entropy < 40:
        strength = "Weak"
    elif len(password) < 12 or entropy < 60:
        strength = "Moderate"
    elif len(password) >= 16 and entropy >= 90:
        strength = "Very Strong"
    else:
        strength = "Strong"

    return strength, entropy


def generate_passphrase(word_count, separator):
    words = [
        secrets.choice(WORDS)
        for _ in range(word_count)
    ]

    number = secrets.randbelow(900) + 100

    return separator.join(words) + separator + str(number)


class PasswordGeneratorApp:

    def __init__(self, root):
        self.root = root
        self.root.title("Advanced Password Security Tool")
        self.root.geometry("800x650")
        self.root.minsize(700, 550)

```

Code (Part 2 of 6)

```
        self.upper = tk.BooleanVar(value=True)
        self.lower = tk.BooleanVar(value=True)
        self.digits = tk.BooleanVar(value=True)
        self.symbols = tk.BooleanVar(value=True)
        self.exclude = tk.BooleanVar(value=False)
        self.show_password = tk.BooleanVar(value=True)
        self.mode = tk.StringVar(value="password")

        self.passwords = []

        self.create_interface()

    def create_interface(self):
        main = ttk.Frame(self.root, padding=15)
        main.pack(fill="both", expand=True)

        ttk.Label(
            main,
            text="ADVANCED PASSWORD SECURITY TOOL",
            font=("Arial", 18, "bold")
        ).pack(pady=8)

        ttk.Label(
            main,
            text="Phase 3 | Secure Password Generator",
            font=("Arial", 10)
        ).pack(pady=3)

        mode_frame = ttk.LabelFrame(
            main, text="Generation Mode", padding=10
        )
        mode_frame.pack(fill="x", pady=8)

        ttk.Radiobutton(
            mode_frame,
            text="Custom Password",
            variable=self.mode,
            value="password",
            command=self.update_mode
        ).pack(side="left", padx=10)

        ttk.Radiobutton(
            mode_frame,
            text="Memorable Passphrase",
            variable=self.mode,
            value="passphrase",
            command=self.update_mode
        ).pack(side="left", padx=10)

        settings = ttk.LabelFrame(
            main, text="Settings", padding=10
        )
        settings.pack(fill="x", pady=5)

        row = ttk.Frame(settings)
        row.pack(fill="x", pady=5)

        ttk.Label(row, text="Password length (8-64):").pack(side="left")

        self.length_entry = ttk.Entry(row, width=7)
        self.length_entry.insert(0, "16")
        self.length_entry.pack(side="left", padx=8)

        ttk.Label(row, text="Number of passwords (1-5):").pack(
            side="left", padx=(15, 0)
        )

        self.count_entry = ttk.Entry(row, width=7)
        self.count_entry.insert(0, "3")
        self.count_entry.pack(side="left", padx=8)

        self.category_frame = ttk.Frame(settings)
        self.category_frame.pack(fill="x", pady=5)

        ttk.Checkbutton(
            self.category_frame,
            text="Uppercase",
            variable=self.upper
        ).pack(side="left", padx=5)

        ttk.Checkbutton(
            self.category_frame,
            text="Lowercase",
            variable=self.lower
        ).pack(side="left", padx=5)
```

Code (Part 3 of 6)

```
        ttk.Checkbutton(
            self.category_frame,
            text="Numbers",
            variable=self.digits
        ).pack(side="left", padx=5)

        ttk.Checkbutton(
            self.category_frame,
            text="Symbols",
            variable=self.symbols
        ).pack(side="left", padx=5)

        self.exclude_check = ttk.Checkbutton(
            settings,
            text="Exclude confusing characters (O, 0, I, l, 1)",
            variable=self.exclude
        )
        self.exclude_check.pack(anchor="w", pady=5)

        self.phrase_frame = ttk.Frame(settings)

        ttk.Label(
            self.phrase_frame, text="Number of words (4-8):"
        ).pack(side="left")

        self.words_entry = ttk.Entry(self.phrase_frame, width=7)
        self.words_entry.insert(0, "5")
        self.words_entry.pack(side="left", padx=8)

        ttk.Label(
            self.phrase_frame, text="Separator:"
        ).pack(side="left", padx=(15, 0))

        self.separator_entry = ttk.Entry(
            self.phrase_frame, width=7
        )
        self.separator_entry.insert(0, "-")
        self.separator_entry.pack(side="left", padx=8)

        buttons = ttk.Frame(main)
        buttons.pack(fill="x", pady=10)

        ttk.Button(
            buttons,
            text="Generate",
            command=self.generate
        ).pack(side="left", padx=4)

        ttk.Button(
            buttons,
            text="Copy Selected",
            command=self.copy_password
        ).pack(side="left", padx=4)

        ttk.Button(
            buttons,
            text="Clear Results",
            command=self.clear_results
        ).pack(side="left", padx=4)

        ttk.Checkbutton(
            buttons,
            text="Show Passwords",
            variable=self.show_password,
            command=self.refresh_results
        ).pack(side="right")

        result_frame = ttk.LabelFrame(
            main, text="Generated Results", padding=8
        )
        result_frame.pack(fill="both", expand=True, pady=5)

        self.results = tk.Listbox(
            result_frame,
            font=("Consolas", 10),
            height=10
        )
        self.results.pack(side="left", fill="both", expand=True)

        scrollbar = ttk.Scrollbar(
            result_frame,
            orient="vertical",
            command=self.results.yview
        )
        scrollbar.pack(side="right", fill="y")
```

Code (Part 4 of 6)

```
        self.results.configure(
            yscrollcommand=scrollbar.set
        )

        self.details = tk.StringVar(
            value="Generate a password and select it to view details."
        )

        ttk.Label(
            main,
            textvariable=self.details,
            wraplength=750
        ).pack(anchor="w", pady=8)

        ttk.Label(
            main,
            text="Passwords are not automatically saved.",
            font=("Arial", 9, "italic")
        ).pack(anchor="w")

        self.results.bind(
            "<<ListboxSelect>>",
            self.show_details
        )

        self.update_mode()

    def update_mode(self):
        if self.mode.get() == "password":
            self.category_frame.pack(fill="x", pady=5)
            self.exclude_check.pack(anchor="w", pady=5)
            self.phrase_frame.pack_forget()
            self.length_entry.configure(state="normal")
            self.count_entry.configure(state="normal")
        else:
            self.category_frame.pack_forget()
            self.exclude_check.pack_forget()
            self.phrase_frame.pack(fill="x", pady=5)
            self.length_entry.configure(state="disabled")
            self.count_entry.configure(state="disabled")

    def get_categories(self):
        categories = []

        if self.upper.get():
            categories.append(string.ascii_uppercase)

        if self.lower.get():
            categories.append(string.ascii_lowercase)

        if self.digits.get():
            categories.append(string.digits)

        if self.symbols.get():
            categories.append(SYMBOLS)

        if self.exclude.get():
            categories = [
                "".join(
                    char for char in category
                    if char not in AMBIGUOUS
                )
                for category in categories
            ]

        return [category for category in categories if category]

    def generate(self):
        self.passwords.clear()
        self.results.delete(0, tk.END)
        self.details.set("")

        if self.mode.get() == "passphrase":
            try:
                count = int(self.words_entry.get())

                if not 4 <= count <= 8:
                    raise ValueError(
                        "Choose between 4 and 8 words."
                    )

                separator = self.separator_entry.get()

                if len(separator) > 3:
                    raise ValueError(
                        "Separator must contain 0-3 characters."
                    )

                phrase = generate_passphrase(count, separator)
```

Code (Part 5 of 6)

```
                self.passwords.append({
                    "value": phrase,
                    "type": "Passphrase",
                    "length": len(phrase),
                    "strength": "Not formally assessed",
                    "entropy": None
                })

                self.results.insert(tk.END, phrase)

                self.details.set(
                    f"Passphrase created using {count} random words "
                    "and a three-digit number."
                )

            except ValueError as error:
                messagebox.showerror("Input Error", str(error))

            return

        try:
            length = int(self.length_entry.get())
            count = int(self.count_entry.get())

            if not 8 <= length <= 64:
                raise ValueError(
                    "Password length must be between 8 and 64."
                )

            if not 1 <= count <= 5:
                raise ValueError(
                    "Generate between 1 and 5 passwords."
                )

            categories = self.get_categories()

            if not categories:
                raise ValueError(
                    "Select at least one character category."
                )

            if length < len(categories):
                raise ValueError(
                    "Length is too short for selected categories."
                )

            for index in range(1, count + 1):
                password, pool_size = generate_password(
                    length, categories
                )

                strength, entropy = check_strength(
                    password, pool_size
                )

                self.passwords.append({
                    "value": password,
                    "type": "Password",
                    "length": length,
                    "strength": strength,
                    "entropy": entropy
                })

                display = (
                    password if self.show_password.get()
                    else "*" * length
                )

                self.results.insert(
                    tk.END,
                    f"{index}. {display} | {strength}"
                )

            self.details.set(
                f"Successfully generated {count} password(s). "
                "Select a result to see its details."
            )

        except ValueError as error:
            messagebox.showerror("Input Error", str(error))

    def show_details(self, _event=None):
        selection = self.results.curselection()

        if not selection:
            return

        index = selection[0]

        if index >= len(self.passwords):
            return
```

Code (Part 6 of 6)

```
        item = self.passwords[index]

        if item["type"] == "Passphrase":
            message = (
                f"Type: Passphrase | Length: {item['length']} characters"
            )
        else:
            message = (
                f"Type: Password | Length: {item['length']} | "
                f"Strength: {item['strength']} | "
                f"Estimated entropy: {item['entropy']:.1f} bits"
            )

        self.details.set(message)

    def refresh_results(self):
        selection = self.results.curselection()
        self.results.delete(0, tk.END)

        for index, item in enumerate(self.passwords, start=1):
            value = item["value"]

            if not self.show_password.get():
                value = "*" * len(value)

            if item["type"] == "Password":
                self.results.insert(
                    tk.END,
                    f"{index}. {value} | {item['strength']}"
                )
            else:
                self.results.insert(tk.END, value)

        if selection and selection[0] < self.results.size():
            self.results.selection_set(selection[0])

    def copy_password(self):
        selection = self.results.curselection()

        if not selection:
            messagebox.showinfo(
                "Select Result",
                "First select a generated password or passphrase."
            )
            return

        index = selection[0]
        value = self.passwords[index]["value"]

        self.root.clipboard_clear()
        self.root.clipboard_append(value)
        self.root.update()

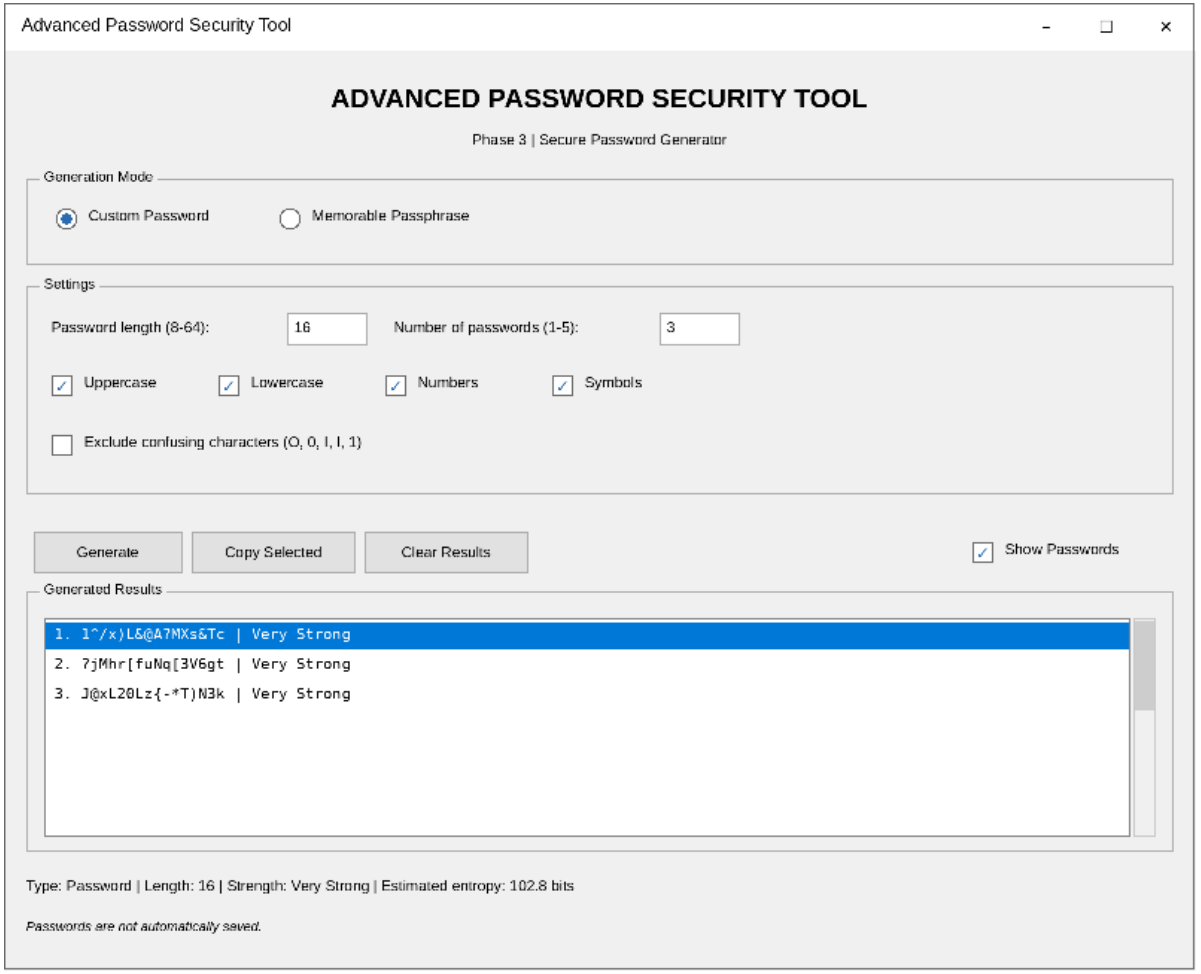
        messagebox.showinfo(
            "Copied",
            "Selected result copied to clipboard."
        )

    def clear_results(self):
        self.passwords.clear()
        self.results.delete(0, tk.END)
        self.details.set("All results cleared.")


def main():
    root = tk.Tk()
    PasswordGeneratorApp(root)
    root.mainloop()

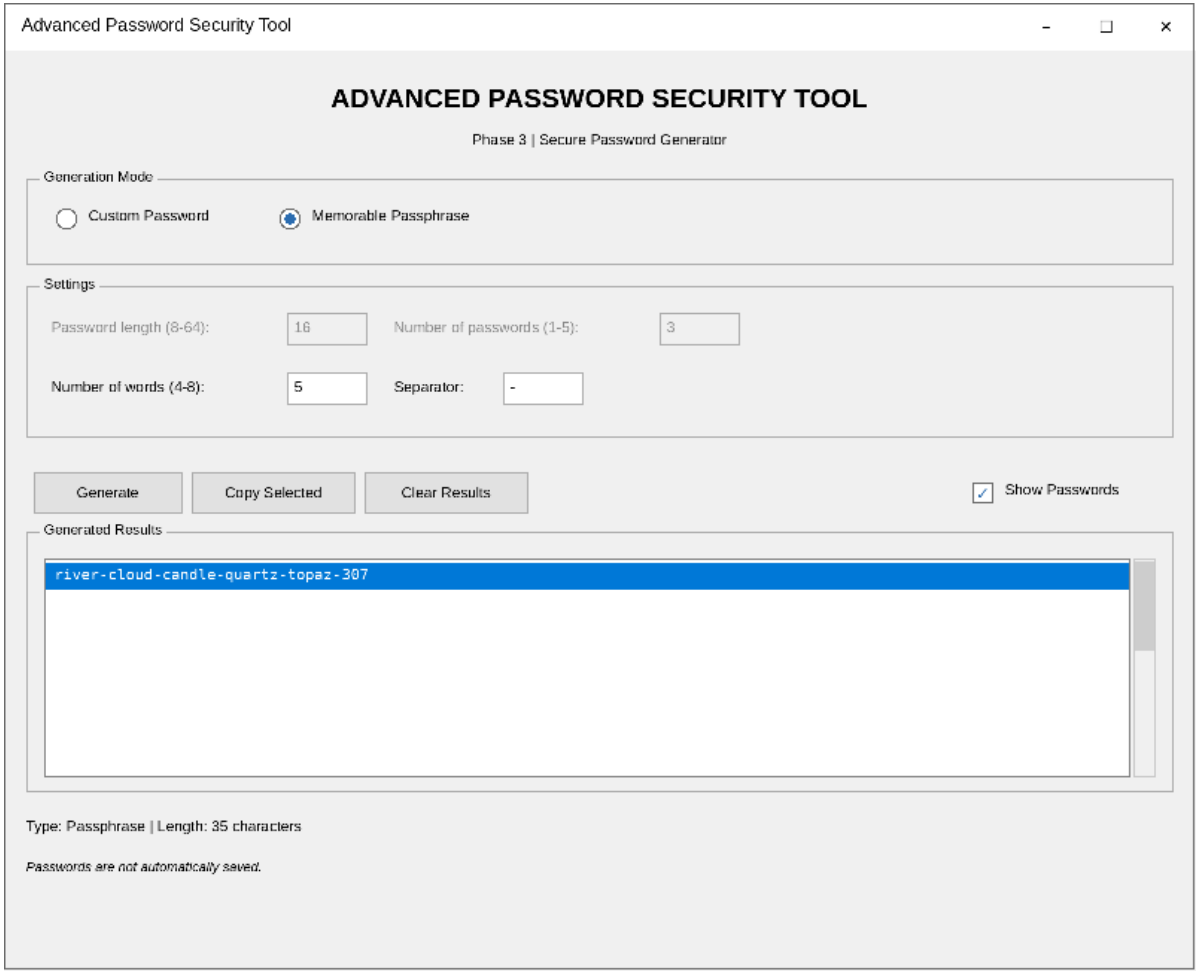

if __name__ == "__main__":
    main()
```

Output - Password Mode



Custom Password mode: 3 passwords of 16 characters, each rated Very Strong (about 102.8 bits)

Output - Passphrase Mode



Memorable Passphrase mode: 5 random words, a "-" separator and a 3-digit number

Future Scope

Use a **larger word list** (e.g. 7776-word Diceware) for stronger passphrases

Add a **formal strength rating** for passphrases

Auto-clear the **clipboard** after a few seconds

Store passwords in an **encrypted vault** with a master password

Check passwords against **known data-breach lists**

Add **dark mode** and package the tool as an **.exe** file

Conclusion

The **Random Password Generator** is a simple and useful Python project.

It helps users create **random and customizable passwords** easily.

It allows users to choose different character types and checks the password strength.

This project helped us learn **Python, classes, functions, input validation and secure random generation** .

In Phase 3, it became a **GUI-based Advanced Password Security Tool** with passphrase mode, entropy-based strength rating, show/hide and copy features.

Thank you